Arquitectura de Computadores

2º Curso Grao Enx. Informática

Práctica 2

Programación Multinúcleo y extensiones SIMD

Objetivos: programar el método de Jacobi en punto flotante, utilizando diferentes grados de optimización (utilización de varios núcleos, utilización de extensiones vectoriales SIMD, estrategias para reducir los fallos caché, etc.) y tomar medidas de rendimiento combinando diferentes optimizaciones y tamaños de problema.

Equipos: grupos de prácticas de dos personas.

Sistema para la toma de datos: FinisTerrae III del CESGA.

Tiempo de trabajo presencial: 5 sesiones.

Plazo de entrega: Para cada equipo **10 días** después de la última sesión de prácticas correspondiente. Comprobar la fecha límite en la tarea correspondiente del campus virtual.

Forma de entrega y formato: Electrónico. Se creará una tarea en el campus virtual para entregarlo. Se hará una única entrega por equipo consistente en una carpeta comprimida en formato ZIP y nombrada como Apellido1Apellido2_Apellido1Apellido2.zip con el siguiente contenido:

- Archivo PDF con la memoria de la práctica (**máximo 6 páginas en total**). El abstract y la introducción no podrán superar la primera página. Hay que seguir el formato de la plantilla disponible en el campus virtual respetando el tamaño de letra máximo.
- makefile and counter.h (**descargados del campus virtual**)
- v1.c, v2.c, v3.c, v4.c con el código fuente de cada uno de los apartados en una única carpeta nombrada “ACP2”.

Valoración de esta práctica:Es el 50% de la nota de prácticas (hasta 2.5 puntos sobre 10 en la nota total).

Objetivo:

Implementar el método de Jacobi, un algoritmo iterativo utilizado para resolver sistemas de ecuaciones lineales. Consiste en partir de una estimación inicial de las soluciones del sistema y luego mejorarla iterativamente. En cada iteración, cada incógnita se calcula usando los valores de las otras incógnitas de la iteración anterior, siguiendo una fórmula derivada del sistema de ecuaciones. El proceso se repite hasta que las soluciones convergen, es decir, las actualizaciones entre iteraciones son lo suficientemente pequeñas, lo que indica que se ha alcanzado una solución aproximada.

Se deberán implementar en C diferentes versiones del algoritmo usando el siguiente pseudocódigo de partida:

Entradas:

```
a: Matriz de coeficientes del sistema (double[n x n])
b: Vector de términos independientes (double[n])
x: Vector solución (double[n])
tol: Tolerancia para la convergencia (double)
max_iter: Número máximo de iteraciones (int)
```

Variables auxiliares:

```
x_new: Vector nueva solución (double[n])
```

Cómputo:

```
Para iter (int) desde 0 hasta max_iter:
    norm2 = 0: norma del vector al cuadrado (double)

    Para i (int) desde 0 hasta n:
        sigma = 0.0 (double)
        Para j desde 0 hasta n:
            Si i ≠ j:
                sigma += a[i][j] * x[j]
        x_new[i] = (b[i] - sigma) / a[i][i]
        norm2 += (x_new[i] - x[i]) ^ 2

    x = x_new
    Si sqrt(norm2) < tol:
        Terminar
```

Salida:

```
Imprimir valor de norm2
```

<u>Las diferentes versiones que hay que desarrollar son las siguientes:</u>

i) Programa secuencial base. El que usaremos como referencia.

ii) Programa secuencial con optimizaciones basadas en el uso de memoria caché.

iii) Programa optimizado paralelizado utilizando programación paralela en memoria compartida (OpenMP).

iv) Programa optimizado paralelizado utilizando paralelismo a nivel de datos (procesamiento vectorial SIMD).

Detalles sobre las diferentes versiones:

i) **Programa secuencial base: (OBLIGATORIO)**

Se trata de implementar en C directamente el pseudocódigo anterior sin ningún cambio ni optimización.

ii) **Programa secuencial con optimizaciones basadas en el uso de memoria caché: (OBLIGATORIO)**

El objetivo de este apartado es realizar optimizaciones sobre el código del apartado anterior de modo que se obtenga al final el mismo vector resultado pero que se reduzca el tiempo de ejecución.

Para ello, implementar las siguientes mejoras:

1. Reducción del número total de instrucciones a ejecutar.
2. División lazos.
3. Desenrollamiento de lazos.
4. Realización de operaciones por bloques.
5. Búsqueda de la mejor combinación de las optimizaciones anteriores.

Esta práctica **debe implementarse estrictamente el método de Jacobi**, no siendo válidas optimizaciones que modifiquen el algoritmo como, por ejemplo, cambiarlo por Gauss-Seidel.

iii) **Programa optimizado paralelizado utilizando programación paralela en memoria compartida (OpenMP): (OBLIGATORIO)**

Por último, realizar otra implementación alternativa, utilizando paralelismo mediante OpenMP. Deben estudiarse los efectos de los siguientes factores:

1. Variación del número de hilos utilizados.
2. Empleo de distintos mecanismos de "scheduling".
3. Comparación de cláusula critical y cláusula de reducción.
4. Búsqueda de la mejor combinación. ¿Eres capaz de mantener los hilos entre iteraciones?

En este apartado solo está permitido usar las funciones pertenecientes al OpenMP versión 3.0 resumidas en: https://www.openmp.org/wp-content/uploads/OpenMP3.0-SummarySpec.pdf

iv) **Programa optimizado paralelizado utilizando paralelismo a nivel de datos (procesamiento vectorial SIMD). (OPTATIVO)**

Realizar una implementación alternativa a la del apartado ii), utilizando las extensiones AVX256 (puedes aprovechar también las FMA), para conseguir paralelismo a nivel de datos.

En la vectorización no se permite el uso de funciones "u" para memoria no alineada

Aquí tienes la lista de funciones: https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html#techs=AVX_ALL

Requisitos de implementación:

El código fuente debe ser implementado usando C estándar. Se permite usar cualquier característica definida hasta el C17, como por ejemplo declarar variables dentro de loops *for (int i=0; i<n; i++).* No se permite usar directivas de GCC, por ello si necesitar alinear memoria no uses *__attribute__ ((aligned (n)))* usa en su lugar la función estándar *_Alignas* o (incluyendo la cabecera <stdalign.h>) *alignas*. Para las reservas dinámicas debes seguir usando _mm_malloc o aligned_alloc.

Cada versión del programa se implementará en un fichero diferente siguiendo el formato especificado en la entrega. Para conservar la legibilidad el programa deberá incluir la cabecera `#include "counter.h"` y no copiar su contenido.

Para el uso de extensiones vectoriales debe incluirse en el código fuente la cabecera `#include <immintrin.h>`. Para el uso de OpenMP debe incluirse la cabecera `#include <omp.h>`.

PARA QUE UNA VERSIÓN SEA EVALUADA, DEBE COMPILAR Y EJECUTARSE HASTA EL FINAL. Un resultado incorrecto o una carrera crítica penalizarán la nota.

El código debe **ESTAR COMENTADO** para indicar qué se hace en cada sección del mismo.

Experimentación:

Para la realización de los experimentos se proporciona un makefile que se encargará de la compilación y ejecución del código. El programa recibirá como argumento n (tamaño) y c (número de hilos). El primer parámetro es obligatorio y deberá comprobarse. El segundo será opcional, solo se deberá comprobar en la versión OpenMP (el en resto será ignorado) y por defecto tendrá el valor 1.

ES REQUISITO INDISPENSABLE QUE EL PROGRAMA ENTREGADO FUNCIONE CON EL MAKEFILE QUE SE OS APORTA. EN OTRO CASO NO SE CORREGIRÁ VUESTRA PRÁCTICA.

Con *make* compilaréis las 4 versiones y con *make run* la ejecutaréis. CFLAGS son los argumentos del compilador y ARGS los del programa, *make* solo compilará si hay cambios en el fichero fuente, por ello podéis usar -B para forzar que recompile. *make* imprime por consola todos los comandos que ejecuta, **FIJAOS BIEN.**

Ejemplos de comandos:

- make CFLAGS="-O2" -> compila todas las versiones con O2
- make run ARGS="1 4" -> ejecuta todas las versiones con n=1 y c=4
- make run -B CFLAGS="-O3" ARGS="1 4" -> compila y ejecuta todas las versiones

El valor de CFLAGS solo se necesita para cambiar el nivel de optimización, el resto de parámetros del compilador ya están fijados, incluido los warnings.

Hacer experimentos considerando que el número **n** toma los siguientes valores: **n=1250 (pequeño), 2000 (mediano), 3200 (grande)**, aunque el programa deberá estar preparado para ser corregido con cualquier otro. La tolerancia se establecerá a **tol**=**1e-5** y aunque ese debería ser siempre el criterio de parada, estableceremos como número máximo de iteraciones

max_iter=15.000. La matriz y los vectores que dependen de **n** deberán reservarse de forma **dinámica,** se permite reserva estática para vectores temporales con longitud fija que no dependa de **n**. La **matriz** y el **vector de términos** deberán ser inicializados con valores aleatorios, para que el método de Jacobi converja, la matriz debe ser **diagonalmente dominante** (el valor de la diagonal mayor que la suma del resto de la fila), lo cual puede garantizarse sumando el total de cada fila a su elemento diagonal.

Medir el número de ciclos de la parte del programa en que se hace la computación indicada en el pseudocódigo (sin contar la impresión del valor final). **Asegurarse de que el resultado final en todas las versiones es el mismo, es decir, que todas las versiones realizan la computación correctamente.** Cuando se utilicen OpenMP o extensiones AVX256 se debe incluir en la medida de tiempo todo lo que implique una sobrecarga respecto del programa secuencial optimizado. Para cada caso, tomar al menos 10 medidas, y seleccionar la **mediana** de estos valores como valor final de medida de tiempo de ejecución.

Memoria de la práctica:

Formato:

Será un documento en PDF siguiendo la plantilla disponible en el campus virtual que contiene los siguientes apartados: título, autores, resumen, **introducción**, descripción de los experimentos, resultados y su análisis, **conclusiones y bibliografía**. Utilizar el mismo tamaño de letra y doble columna para la edición del texto.

Contenido de la memoria:

Es un informe técnico. Hay que explicar lo más relevante de los códigos asociados a cada apartado incluyendo un pseudocódigo, incluir gráficas de resultados e interpretación de dichos resultados. La memoria debe incluir las gráficas que se consideren oportunas para sacar conclusiones de los resultados obtenidos.

Es de especial interés representar la ganancia en velocidad (también llamada aceleración o *speedup*) de la versión secuencial optimizada con respecto a la versión inicial compilada con -O0, la ganancia en velocidad de los códigos de los apartados iii) y iv) con respecto a la versión secuencial optimizada, y finalmente las versiones desarrolladas en los apartados ii), iii) y iv) respecto de la versión inicial compilada con -O3).

En el caso del apartado iii), además de las gráficas que se consideren adecuadas, es imprescindible representar en una gráfica para cada valor de N la ganancia en velocidad conseguida.

Criterios de evaluación:

Cumplir las especificaciones del enunciado de la práctica buscando siempre obtener la mínima latencia (tiempo de ejecución) del programa en las diferentes implementaciones, rigurosidad en el planteamiento e interpretación de resultados, exploración de alternativas, estudio autónomo, presentación de resultados (calidad de la memoria) y entrega en el plazo marcado.